

Technical Paper

Inside FactBank: Architecture, Retrieval, and Where the Gains Come From

Paper II: The Technical Design: components, data flow, algorithms, key decisions, and measured advantages

By Mohannad. This paper was drafted by an AI assistant, because I do not yet know how to write a formal research paper. The FactBank project, its ideas, decisions, and solutions, are mine; the AI helped with the coding, the brainstorming, and with writing this paper. It is grounded only in the project's own files, and where something is unknown it is marked as such.

1. Overview

FactBank corrects a language model’s stale library knowledge by delivering curated, version-tagged “landmine facts” at inference time. The design principle is a strict division of labour: the model’s weights (the **fuzzy core**) do reasoning and code structure; a read-only **frozen bank** of facts supplies exact post-cutoff knowledge; and a **code-driven loop**, not a model tool-choice, decides to retrieve and places the results in front of the model before it answers. The system’s most-shipped form places the entire mechanism inside two metadata keys of a GGUF model file, so it runs wherever the model runs, with no companion service.

The project now ships **four experts** — three security/networking departments (netsec, offsec, dataplane) and a large **application-security** expert whose bank makes the model write secure code without being asked. The appsec expert is the reason several mechanisms in this paper grew a new branch (a faceted fact schema, a second delivery route past a loader size cap, and a working fix for the thinking-on regression), and it is the first FactBank expert benchmarked against an external, third-party security suite.

2. One shipping bake, one legacy bake, and a Python test bench

A crucial technical clarification, made explicit in the repository’s ARCHITECTURES.md, is that three different retrieval systems appear in the project and are all loosely called “the FactBank bake”, but they are **not peers**. Exactly one is the current, shipping bake; one is a **legacy** bake method that has been superseded; and the third is **not a bake at all** but a Python test bench used to validate facts and the retrieval algorithm without writing anything into a model. Keeping them apart matters, because the system that produced the headline benchmark numbers is the shipping bake, while the richer soft-door / pointer / MMR recipe that appears elsewhere in the docs belongs to the test bench, which is Python-only and never ran in-engine.

System	Status	Where it runs / what it does	Role
Baked-Index (bake_index.py)	CURRENT, shipping	Inside the GGUF chat-template,	Produced every published model and

		executed by llama.cpp; hard library gate + inverted-index term match; injects top-N	all base→baked benchmark numbers
Static-Bake (package/factbank/bake.py)	LEGACY, superseded	Bakes ALL facts into one static system prompt; no retrieval, gate, or ranking, every fact always present	Older whole-bank route; does not scale (every request carries the whole bank), replaced by Baked-Index
Served-Loop / lookup.py	TEST BENCH, not a bake	A pure-Python retrieval prototype (draft → search → re-derive): soft doors, pointer chains, MMR, double-key (HyDE)	Validates facts and the retrieval algorithm offline, without baking into a model; produced the Python hand-read results and Qwen probes

The remainder of this paper describes the shipping **Baked-Index** system in detail (it is the one that was benchmarked and shipped), then covers the fact-making pipeline that feeds all routes. The **legacy Static-Bake** and the **Python test bench** are noted only where they differ. Where a claim belongs to only one system, that is stated. Section 6.2 makes one such distinction explicit and load-bearing: the shipped bake retrieves on the **prompt only**, while the test bench retrieves on the prompt **and** the model's own draft (HyDE double-key). That difference sets a ceiling the shipped artifact does not reach, and this paper labels served-loop numbers as a **method ceiling**, never as the shipped result.

3. Making the facts: the extraction pipeline

Facts are produced by an LLM-first structured extractor in which **the model does the meaning and dumb code verifies every field**. The pipeline is FIND → extract → repair → check.

3.1 FIND, source targeting (the one law)

Before any extraction, the builder must obtain the library's official **migration / breaking-changes guide**, not release notes, not a blog. That is where a project documents its breaks. The rule is absolute: a perfect extractor pointed at the wrong document yields a bank of the wrong facts. A library with no prose upgrade doc is deemed un-mineable (the reason kymatio, lightgbm and tqdm are left uncovered). The law also applies **inside** a source: when a single source is huge (MITRE CWE, a SAST rule set), the builder curates within it and gates the low-bite material rather than swallowing the whole file.

3.2 Extract, structured, function-calling, code-derived fields

The default backend is DeepSeek (deepseek-v4-flash) with reasoning disabled, called via **function-calling rather than a strict JSON-schema** (DeepSeek has no strict json_schema). The source is chunked (3,500 characters, split on blank lines) and each chunk is one API call. The model fills a strict schema:

Field	Meaning		
type	REMOVED	REPLACED	CHANGED (enum)
subject	the API / symbol the fact is about		
old	the exact dead or changed symbol		
new	the replacement, or null		
truth	one concrete, actionable sentence (stop X, use Y / you now get Z)		
why_it_bites	post-cutoff	reverses-habit	silent-failure (enum)
quote	a VERBATIM span copied from the source, the anchor		
associative	3-8 short phrases a developer would TYPE that are not already in the fact		

Certain fields are **never trusted to the model and are derived in code**: the fact type is reconciled from whether a replacement is present; provenance keywords are pulled from old/new/truth; the id is a hash of type|subject|old|new; and the library and version are stamped. The project reports that deriving these in code rather than trusting the model's enum alone moved one source from 35% to 96.5% clean, a concrete demonstration that the "design over output-mode" decision is load-bearing.

For the application-security expert the extractor was extended so that a fact can **carry its own code**. Two optional verbatim fields, code_bad and code_good, hold the insecure and secure snippet directly, copied from the source **modulo whitespace only** and never passed through the identifier-squashing canonicaliser (canonicalising code would corrupt it). This removes a separate code-search step: the example travels inside the fact.

3.3 Repair and check, the anti-hallucination gate

A repair step re-grounds any paraphrased quote back to a real source line (it requires at least one fact-symbol in the line and at least two shared tokens). The check step then enforces the **verbatim-quote anchor**: a fact whose quote is not a substring of the canonical source is dropped to a rejects file. The quote must be at least 12 characters; single-token "old" symbols are

additionally required to appear in the source. The application-security pipeline strengthens repair to **sentence-boundary** grounding (whole real sentences, not ellipsis-joined fragments) and dedupes across its seven sources on **prose only** — a shared code_bad snippet is not treated as a duplicate signal, because two different weaknesses legitimately share an example.

The project is careful about what this gate does and does not prove. It proves a quote is real; it does not prove the fact is still true for the current version. Migration guides are historical, and a documented change can be reverted, so habit-reversal facts about defaults must be re-checked against the library’s current source before shipping. A real example was caught and removed: an ldap3 changelog said the default connection strategy changed from SYNC to RESTARTABLE, but the current default is SYNC (reverted), shipping it would have taught the model the wrong answer.

Because grounding proves a quote is REAL and not that it is CORRECT, the application-security bank added a **mandatory adversarial correctness audit**: a second model re-checks each fact against its claim, and it removed **~3.8% wrong or stale facts** that were perfectly grounded yet false for the current version. Grounding is the floor of trust, not the ceiling.

4. Storing the bank

The master copy of a bank is plain JSONL, one fact per line, carrying an id, the injected text, a source, a version, a kind, and metadata (including the source URL). Facts have two provenances, and the distinction drives everything downstream:

Kind	Origin	Worth
curated	Hand/agent-extracted from real docs, gated on a verbatim quote	High, renames, removals, gotchas: the facts that beat a wrong prior
mined	inspect.signature over the installed library plus its Sphinx objects.inv doc URL	Unproven, mined facts have won zero eval cases; hallucination-proof but shallow

Provenance is encoded in the id (mined ids contain “api-”), and at bake time curated facts are sorted first so that “is this curated?” becomes an integer comparison costing zero runtime bytes. The project is candid that the large mined pool (tens of thousands of facts across dozens of libraries) is roughly two-thirds junk because the miner walks the inheritance chain and donates inherited methods to the wrong class; the junk flattens IDF and crowds the top-k, and is the likely cause of gold retrieval falling from 12/12 at ~2.3k facts to 9/12 at 21k facts. The stated conclusion, **a bigger mined pool is not an upgrade**, is an honest technical position, not a marketing one.

One property makes the measurements meaningful: every fact targets a **post-training-cutoff API**. With an old, well-known library you could never separate “the bank told it” from “it already knew.” Because the facts are all post-cutoff, any correct answer must have come from the bank.

4.1 The faceted concept→variant schema (v3)

The application-security expert introduced a second, richer bank layout, because a flat “one library, one fact” shape does not fit a domain organised by **weakness** rather than by library. The shipped bank, `FINAL_v3.jsonl`, is faceted: **258 concepts (254 of them MITRE CWE weaknesses, plus 4 synthetic `door:\*` grouping concepts)** expand into **3,984 variants**, of which **1,075 carry a verbatim bad/good code pair**. A concept is the weakness (“XML external entity”, “deserialising untrusted data”); a variant is one concrete instance of it in one language or framework, with its own quote, its own code, and its own feature phrases.

This bank is **not a short library list**. It spans **10+ languages** — Python (1,315), Java (729), JavaScript (360), C (290), Swift (230), Go (191), C# (139), Rust (98), Ruby (97) and more — across frameworks including Android, iOS/SwiftUI/WKWebView, Flask, Django, Express, Node, ASP.NET/.NET, Spring, Java EE, Rails and Laravel. It draws on **seven permissive sources**: MITRE **CWE**, GitHub **CodeQL**, SAST rules (**Bandit** + **gosec**), OWASP **MASTG**, **RustSec**, **NIST/RFC/Mozilla** crypto-net guidance, and **OWASP**.

The facets exist to serve the **benign-prompt bridge**. An application-security landmine must fire on a prompt that never names the vulnerability — a developer asks “load this config file”, not “avoid CWE-502”. So each variant is indexed by its **coding task**: feature phrases and use cases (“load a YAML config”, “parse an uploaded XML”, “download a model checkpoint”) that a benign request actually contains. The concept→variant split lets one weakness carry many such bridges without duplicating the fact. The insecure-by-default landmines this produces — `torch.load(weights_only=True)` (a post-cutoff default), `XXE resolve_entities=False / no_network`, `yaml.safe_load` over `yaml.load`, `secrets` over `random`, parameterised SQL, `os.environ` credentials over hardcoded ones, `ast.literal_eval` over `eval`, constant-time HMAC comparison — are the facts that make a stock model write secure code without being told to.

5. Baking: building the in-template retriever

The bake step (`bake_index.py`) turns a bank into an inverted-index retriever spliced into the model’s chat-template. Each fact becomes a set of weighted index terms, where the weight expresses how strongly a word proves that this fact is the one wanted:

Term source	Weight
dead API name (renamed/removed, rare)	10
rare API token	4
description word (curated fact)	3
task-phrase bigram	3
task-phrase word	2
Doc2Token expansion bigram	2
expansion word / common token	1

Four transformations are then applied, each present because its absence broke something concrete:

- **Squash-normalisation and identifier splitting**, `ttest_ind` is also indexed as `ttest`, `ind` and `ttestind`; scikit-learn as `scikitlearn`. This is what lets a user's "t-test" reach `scipy.stats.ttest_ind`. Only strong (weight ≥ 4) tokens earn variants; squashing every description word blew the index from 11k to 18k terms.
- **Doc2Token expansions**, the model wrote thousands of everyday questions for its own curated facts, offline; their bigrams are indexed at a weight strictly below real terms. This cures the hardest keyword-retrieval failure: a question that shares zero words with its fact ("turn columns into rows" now finds the `melt`→`unpivot` fact).
- **IDF at build time**, a term pointing at ≤ 3 facts is evidence ($\times 3$); ≤ 10 , weak ($\times 2$); more, noise ($\times 1$); above 40 postings it is dropped. Without this, "test" (in every SciPy docstring) outvoted "ttest." The multiplier is applied at build time so the runtime stays pure integer addition.
- **Control culling**, any term that appears in the control questions (e.g. "write me a haiku") is deleted from both the index and the gate, so a benign non-coding prompt opens no tab and injects nothing.

The output is stored as a single dictionary mapping **term** → "**factid:weight factid:weight ...**". The build is fully deterministic, every iteration is sorted and every JSON blob uses `sort_keys`, because Python's per-process hash randomisation otherwise produced byte-different but semantically identical templates, violating the repository's rule that a shipped artifact must be regenerable from source.

5.1 The gate and its aliases

Beyond the index, the bake builds a **gate**: per library, its name plus up to 48 unique trigger words plus verified aliases (e.g. `np` → `numpy`). A named library always wins; inferred tabs open only when nothing is named. A gate trigger must be unique to one library and must not be a word any control question uses, or it would open the wrong tab (or a tab on a haiku).

A real defect and its fix are recorded. The in-engine gate opens on the exact library token, so a user typing a **natural or old name** ("BloodHound" not `bloodhound-py`, "CrackMapExec" not `netexec`) opened no tab → zero facts injected → the model answered from stale training and missed. The fix (`gen_gate_aliases.py` → `bake_index.py --extra-aliases`) derives per-expert aliases from each rename fact's old name plus the library's natural stem, establishing the general law: **every rename fact's old name is a gate alias, it is exactly what a stale user types**. All published security experts were re-baked with this fix. (For the faceted application-security bank, which is organised by weakness rather than by library, retrieval routes on the coding task through feature phrases rather than through a library gate; the library gate is loosened accordingly so a benign prompt that names no library still opens the relevant weakness.)

5.2 Writing it back into the model

The baker rewrites the GGUF's `tokenizer.chat_template` with the index plus the retriever, and also carries the raw JSONL in a `factbank.bank` metadata key for auditability. All weight tensors are copied byte-for-byte; only metadata changes, so the output GGUF is a full copy of the model with untouched weights. A hard requirement learned the expensive way: llama.cpp probes the chat-template at load time with synthetic inputs to auto-detect the tool-call format, and a template that throws on any probe makes it reject the entire model, so every code path in the template is guarded.

5.3 The size wall and the two delivery routes

There is exactly one residual size wall, and the project is precise that it is not a format or engine limit: **LM Studio's raw-GGUF loader** silently swaps a chat-template over ~980 KB for a 48-character sentinel, so the model loads "fine" and answers garbage. llama.cpp itself caps at 1 GiB and errors loudly. The guard survives only as an opt-in hard-fail for that one route. A ~100-fact bank is about 115 KiB; a ~4,000-fact bank about 2.7 MB.

The application-security expert crossed that wall. Its faceted template is **4.18 MB** — well over the ~980 KB raw cap — so it is the first expert to ship on the **LM Studio Hub `model.yaml` route** instead of raw-embedded metadata. On that route the full template is delivered to the engine via `llm.load.promptTemplate (--chat-template-file)`, which has **no size limit** (proven earlier at 1.5 MB and 2.0 MB, and now at 4.18 MB). The Hub `model.yaml` also carries the serving settings that a raw GGUF cannot force: `config.operation.fields` bake the native sampling (temperature 1.0, topK 64, topP 0.95, minP 0.01) and `config.load.contextLength` sets 32768, so the model arrives correctly configured rather than depending on a client remembering to set them. So there are now two delivery routes: raw-GGUF metadata for small banks, and the Hub `model.yaml` route for banks whose template exceeds the raw cap.

6. Retrieval at inference, the data flow

For the Baked-Index system, the following runs inside llama.cpp's Jinja engine on the last user message, **before the question renders**:

- **Normalise**, lowercase, punctuation to spaces.
- **Gate**, if the user names a library, only that tab opens (named always wins); otherwise tabs open on trigger words. No tab → nothing renders, and the request is byte-identical to a stock model's.
- **Build query terms**, content words, their squashed/split variants, alias canonicals, and adjacent bigrams.
- **Look up each term in the postings dictionary**, a fact is never visited unless a term points at it; hits outside open tabs are discarded.
- **Score**, integer additions; curated facts get +6, or +12 if the question shows breakage intent ("broke", "stopped working", "after upgrading").

- **Take the top 5** by max-selection.
- **Deliver** via the tool-response lane (below).

6.1 The delivery lane, a forged native tool exchange

Facts are not handed over as prose. The template renders a **completed factbank_search call and response in the model's own tool syntax**, bit-identical to a real tool exchange, using the base template's own macro. The model's experience is: a documentation lookup already ran; here are the results; now answer.

This choice is empirically motivated. Handed the same facts as prose, the model deliberates, one glyph-heavy question burned 8,189 reasoning tokens and produced no answer. Through the tool channel, the same question took 390-757 tokens and returned a correct answer: a tool_response is not something the model argues with. The block is rendered before the user's question, which both fixes an echo bug and buys prompt-prefix caching (repeat questions on the same topic drop from ~1.8 s to ~0.24 s to first token).

6.2 Served retrieval vs shipped retrieval (a ceiling, not a result)

The shipped baked GGUF retrieves on the **prompt only**, in-template. The Python test bench (the served loop) retrieves on the prompt **and** the model's own draft — **HyDE double-key**: the model drafts an answer, that draft names the insecure API a benign prompt omits, and the retriever keys on both. The draft key is the single biggest lever for benign application-security prompts, and it **cannot run inside a chat-template** (the template renders once, before generation; there is no draft yet). So the served loop reaches a **higher ceiling than the shipped artifact**, and the two must never be conflated.

Concretely, on the served loop the application-security bank lifted a base model from 14 to **25** secure on the 28-task SecurityEval pattern-subset, and an e2b served bank reached **22** vs DeepSeek-V4's 13 on a 24-task subset. **These are the retrieval-method CEILING (served), not the shipped GGUF.** The shipped, prompt-only numbers are lower and are reported separately in §9. Naming and quantifying this gap — served HyDE double-key vs prompt-only baked — is a deliberate honesty item, not a footnote.

7. Serving decisions that matter

Two settings decide whether a served expert works, and both are documented as traps that cost real time:

- **Native sampling**, use the model's native settings, not a bare low temperature. For Gemma: temperature 1.0, top_k 64, top_p 0.95, min_p 0.01. The min_p floor is what prevents a reasoning model from falling into a **repetition** loop and returning an empty answer, a failure that looks like a token-budget bug but is really a sampling bug. (min_p fixes the repetition loop; it does **not** fix the reasoning spiral of §8 — a different failure mode.) On the Hub

model.yaml route these are baked into the model's settings so the client cannot forget them.

- **The shared token budget**, max_tokens is one budget for reasoning and then the answer. Blow it and you get an empty answer with finish_reason=length and no error, so it must be set large and the context at least as big.
- **KV-cache precision**, never quantise the KV cache on sub-1B models (Q4 KV produces word salad); weights at Q4/Q8 are fine.

8. The reasoning paradox and the authority cure

The project's sharpest technical finding is that enabling chain-of-thought **reduces** the bank's benefit. On the baked 12B model, thinking-off scores 44.2% on the buildable GitChameleon set and thinking-on drops to 36.9% (-7.3). Two mechanisms account for the loss:

- **Reasoning spirals and truncates**, on ~30% of problems the model restates its reasoning without closing the think channel, exhausts the budget, and returns empty; raising the budget only makes it spiral longer. Pass rate is strongly monotone in reasoning length (under 4k chars → 73% pass; over 16k → 14%).
- **Reversion to the trained habit**, a landmine fact is by construction a habit reversal, so when the model deliberates from its parameters the learned habit reasserts itself and argues the injected fact away. This is proven by convergence: the baked+thinking answer is identical to the base model with no bank at all (e.g. falcon .body vs .text; pandas groupby observed default).

The prescribed cures are: ship such banks **thinking-off**; or, for thinking-native models that cannot disable reasoning, prepend an **authority directive** that frames the facts as verified, version-correct, and superseding the model's training. In a controlled probe on the hardest landmine, framed authority held fact-adherence at 18/18 across three delivery channels (system prompt, forged tool-response, both), while an unframed control reverted, with the important caveat, which the project flags itself, that the 0/6 unframed control was read from a session but not retained as a log, so it is illustrative rather than measured. A two-pass strategy (answer thinking-off, then re-ask only the misses with authority + thinking-on) lifted the 12B from 44.2% to 54.2%.

8.1 Making thinking-on actually work: the two-bug fix

Until this session, "ship thinking-off" was the only reliable answer. The application-security expert needed a thinking-on edition as well, so the failure was traced to root cause — and it turned out to be **two stacked bugs**, not one intrinsic limit.

- **Bug (a): a llama.cpp Gemma-4 generation-prompt template bug**. In the generation-prompt path, with thinking **off** the template emitted a **closed** empty thought block, but with thinking **on** it emitted **nothing** — the model was handed no open thought channel, so it never entered a proper reasoning state and instead spiralled until the token cap, producing

empty or truncated answers. The fix opens the channel when thinking is on: the prompt now emits `<|channel>thought\n` so the model starts inside a real, open thought block.

- **Bug (b): weak authority framing.** The injected note’s framing was too soft, so the model reasoned back to its prior and reverted the fact (the §8 reversion mechanism). The fix restores the strong frame proven in the served test — *“AUTHORITATIVE ... MANDATORY ... OVERRIDE your training defaults and any conflicting habit.”**
- **The third piece: force the template default.** Because **LM Studio drops** ``chat_template_kwargs``, an `enable_thinking` client flag has no effect there. So thinking-on is baked as the **template default** (`enable_thinking=true`) rather than left to a client flag that the loader will silently discard.

Measured result (12B, same bank): empty answers **5/30 → 3/30**, truncated **12/30 → 4/30**; the responses now close cleanly, at roughly 2–2.8k reasoning tokens with no truncation. An e2b thinking-on smoke test came back 8/8 clean. A **residual ~10% of the hardest-reasoning prompts still spiral to a blank answer** even at a 12k-token budget; that residual is inherent to reasoning-on, and it **fails safe** — the output is a blank, never insecure code. To be precise about a nearby trap: `min_p 0.01` fixes the low-temperature **repetition** loop, but it does **not** fix this reasoning spiral; they are different failure modes and the spiral needs the channel-open fix, not a sampling floor.

Decision: ship BOTH editions. The two editions differ **only** by the `enable_thinking` default. Thinking-off is the fast, reliable default; thinking-on adds a visible reasoning trace at the cost of that ~10% fail-safe residual. Each shipped repository carries both, with notes telling a user which to pick.

9. What the technical gains actually are

9.1 Capability, correctness lift with no training

On the security experts’ own landmine questions, base→baked lift is large at every size, and error-closure (the fraction of the base’s wrong answers the bank fixed) rises with model size even as raw lift shrinks:

Model	netsec (/48)	offsec (/44)	dataplane (/47)	error-closure trend
2B edge	19 → 32 (+13)	12 → 37 (+25)	5 → 32 (+27)	netsec 45% → 57% → 73%
12B	27 → 39 (+12)	12 → 43 (+31)	16 → 41 (+25)	offsec 78% / 97% / 96%
26B MoE	37 → 45 (+8)	17 → 43 (+26)	20 → 43 (+23)	dataplane 64% / 81% / 85%

The three prior security experts remain shipped as measured (netsec 114 facts / 7 libs, offsec 489 / 17, dataplane 318 / 7).

On the execution-scored GitChameleon 2.0 benchmark, baking a 4,167-fact bank lifted pass@1 by +6.4 points on the 12B (37.8% → 44.2%) and +2.8 on the 26B (43.4% → 46.2%) with no weight changes, on the 249 of 328 problems that build on the local harness (thinking-off). The project is explicit that these are **local-harness**, **non-Docker** numbers, internally fair (base and baked on the same 249) but not directly comparable to the official Docker leaderboard, and it does not claim to beat frontier cloud models. The claim is narrower and well-hedged: a bank turns a small local model into something that scores like a much larger one.

9.2 The application-security expert against an external suite

The application-security expert is the first FactBank expert scored on a **third-party** benchmark: **SecurityEval** (s2e-lab, MSR 2022), 121 Python CWE tasks. The common pattern-judgeable subset — the 21 tasks where every arm is machine-scoreable — was run on the **shipped baked GGUFs**, **thinking-off**, and hand-verified (patterns triage; wins read by hand). DeepSeek-V4 (deepseek-v4-flash) was run in **no-thinking mode** on the same prompts, for a like-for-like cloud comparison. These are **shipped-artifact** numbers, not the served ceiling of \$6.2.

model (baked, thinking-off)	secure / 21
e2b base	11
e2b + bank	13
12b base	17
12b + bank	17
26b base	17
26b + bank	19 (best)
DeepSeek-V4 (cloud, no-thinking)	14

Head-to-head against the cloud model, the bank is **26b +5/-0** and **12b +3/-0**, while e2b is **+2/-3**. So **12B+bank and 26B+bank beat the cloud model, and e2b+bank is roughly level with it**. On each model's own judgeable set the per-size lift is **e2b 15→18, 12b 29→31, 26b 30→33** (+2-3), with a clean XXE (CWE-611) sweep on the bigger models (12b 5→6/6, 26b 4→5/5).

The honest reading, which the project states plainly: a bigger model is more secure to begin with, and the bank helps most where the base is weakest and on specific weaknesses (XXE, deserialisation, weak randomness, hardcoded credentials). The bank fires on 116/121 tasks, so it is genuinely active; the modest e2b lift is a real 2B-obedience limit (a small model does not always follow the injected fact), verified by hand, not a retrieval failure. And the decisive single win is on-thesis: the post-cutoff `torch.load(weights_only=True)` default, which a training-frozen model cannot know.

9.3 Performance, the engineering payoff of the index

The inverted-index design was chosen against a simpler linear scanner and the measured differences are substantial:

- **Retrieval quality and speed**, on the same 2,560-fact bank the index scored 9/12 gold vs the scanner's 6/12, at 809 KB vs 949 KB, and 545 ms vs 2,629 ms per matched request.

- **Data-structure choice**, storing facts in a list (integer ids) rather than a dict measured 291 ms/request vs 1,318 ms, because the template engine builds lists fast and dicts slowly.
- **Caching**, rendering the retrieved block before the question buys prompt-prefix caching: repeat topic questions drop from ~1.8 s to ~0.24 s to first token.

9.4 Cost and portability

Extraction is cloud, no GPU, and costs about four cents per library; the guidance is to extract generously rather than ration API calls. Delivery adds **no runtime cost and no service**: there is no server, no embedding model, no vector store, no client code, and no tool-choice step. Everything that could drift at runtime, the expansions, the IDF weights, the aliases, the gate, is moved to build time, leaving only string operations and integer addition at inference, which is precisely why the whole system fits inside a file that every serving stack already executes.

10. Summary of key design decisions

- Separate knowledge from reasoning; never ask the weights to recall a post-cutoff fact.
- Derive trust-critical fields in code, not from the model (design over output-mode); let a fact carry its own verbatim code when the domain needs it, but never canonicalise that code.
- Ground on a verbatim quote, then AUDIT for correctness — grounding proves a quote is real, not that it is current.
- Retrieve deterministically in code, not via model tool-choice; retrieve first, filter second, fail open.
- Facet by weakness and index by the coding task when the domain is not library-shaped (the benign-prompt bridge).
- Move all drift-prone computation to build time; keep runtime to integer addition and string ops.
- Deliver via a forged native tool-response, before the question, so the model applies rather than arbitrates.
- Keep served (HyDE double-key) results separate from shipped (prompt-only) results; report the ceiling as a ceiling.
- Ship thinking-off for habit-reversal banks; where thinking-on is needed, open the thought channel + force the template default + strong authority framing, accept the ~10% fail-safe residual, and ship both editions.
- Deliver past the raw-loader size cap via the LM Studio Hub `model.yaml` route, with sampling and context baked into the settings.